

What is Generative AI

- Generative AI is a type of artificial intelligence that creates new content, such as text, images, music, and code, by learning patterns from vast amounts of training data.
- Generative AI is trained on large datasets like text, images, audio or video using deep learning networks. During training, the model learns parameters (millions or billions of them) that help them predict or generate content. Here models generate output based on learned patterns and prompts provided.

Main Types of Generative AI Models



**Generative
Adversarial
Network (GAN)**



**Transformer-
based Model**



Diffusion Model

Key Applications

- **Content Creation:** Generating articles, marketing copy, and creative writing.
- **Software Development:** Assisting with code generation, debugging, and documentation.
- **Art and Design:** Creating images, digital art, 3D models, and architectural designs from text prompts.
- **Healthcare and Life Sciences:** Accelerating drug discovery by modeling protein sequences and analyzing medical data.
- **Customer Service:** Powering chatbots and virtual assistants that provide human-like responses.
- **Data Analysis:** Processing large datasets to uncover insights and trends or generating synthetic data for training other AI models.

Benefits of GenAI

Some of the benefits of generative AI include:

- **Speed:** GenAI tools work quickly to complete tasks and generate content.
- **Multiple use cases:** GenAI can be used in several different situations, such as for coding or writing a resume.
- **Scalable:** GenAI tools are often equipped to handle multiple users and tasks without degrading performance.

Limitations of GenAI

Some of the limitations of generative AI include:

- **Requires human oversight and fact-checking:** Because GenAI can make mistakes, lacks a true understanding, and does not have genuine empathy, it requires humans to review its outputs.
- **Bias in training data:** AI systems learn from existing human-created content. If that content contains societal biases or prejudices, the AI can absorb and repeat them.
- **Privacy concerns:** There can be security risks. AI systems store user interactions and inputs, and sensitive personal or business information may be shared during conversations.

Transformers

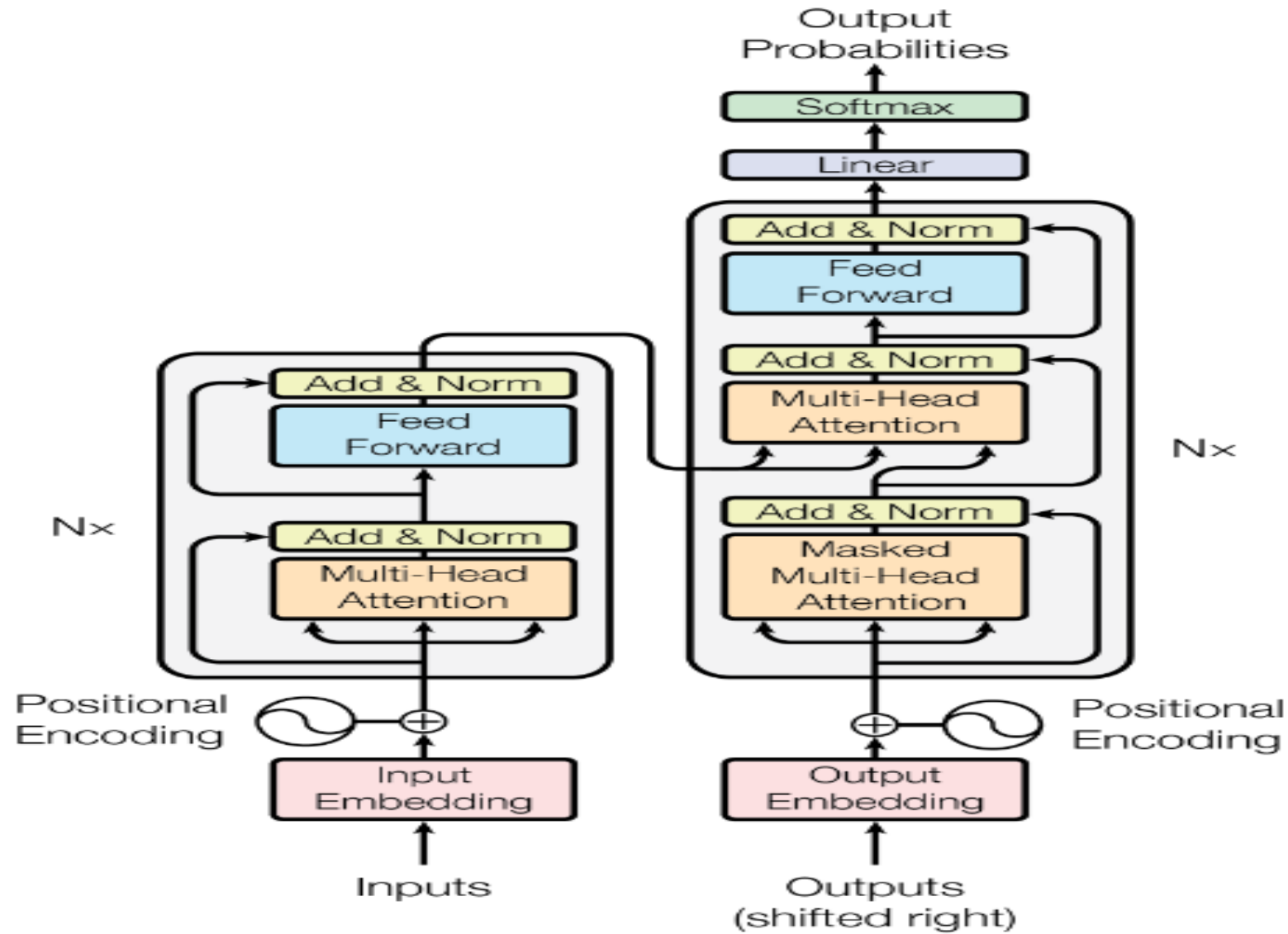


Figure 1: The Transformer - model architecture.

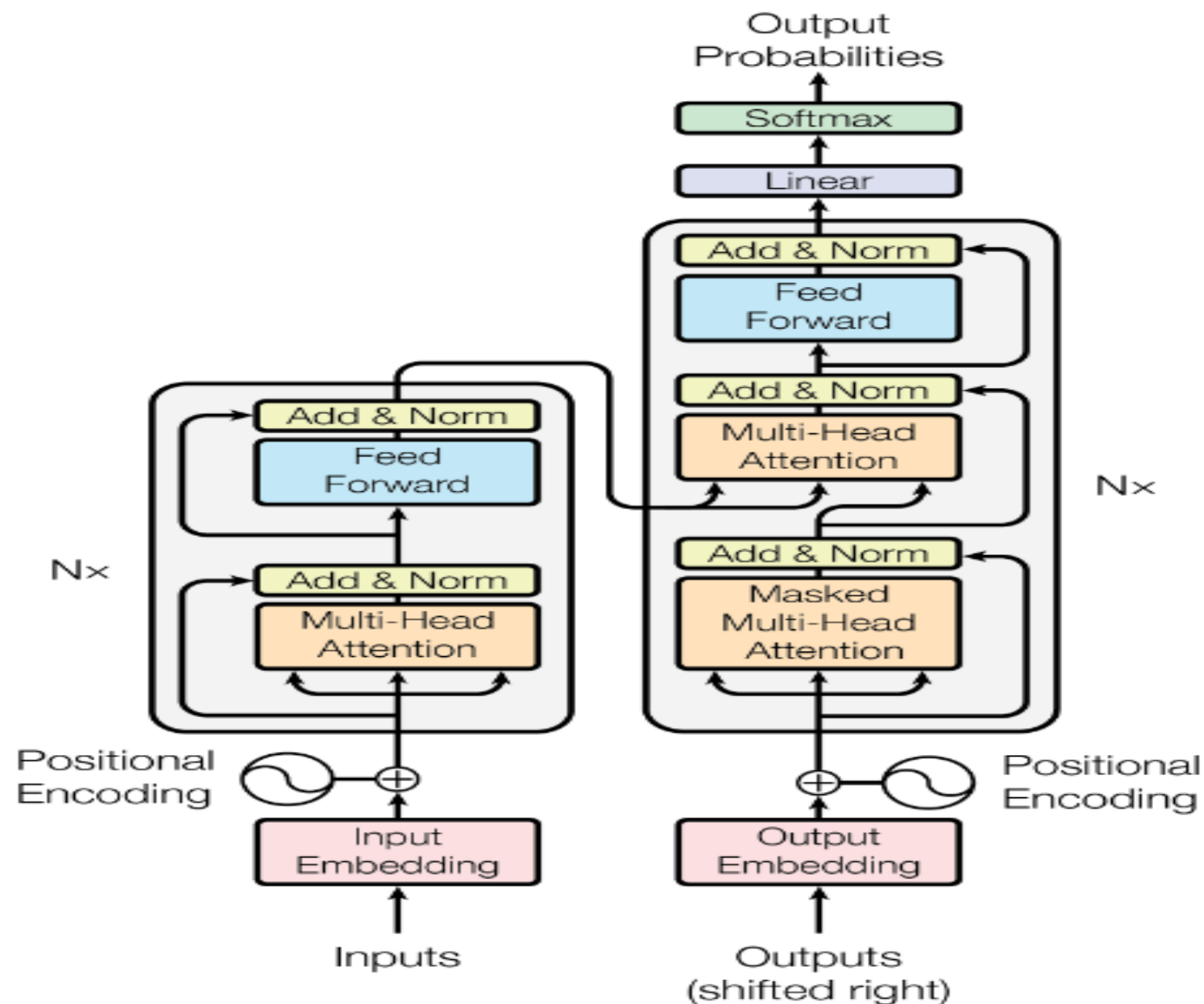


Figure 1: The Transformer - model architecture.

Vectors

What

Ordered set of numbers that represent objects


$$\begin{pmatrix} 2566 \\ 3 \end{pmatrix}$$

Square
footage of
house

Number of
Bedrooms

Vectors

Why



$$\begin{pmatrix} 2566 \\ 3 \end{pmatrix}$$



Vectors

Why



Vectors

Why

$$\begin{pmatrix} 4310 \\ 5 \end{pmatrix}$$



$$\begin{pmatrix} 2566 \\ 3 \end{pmatrix}$$

$$\begin{pmatrix} 4300 \\ 5 \end{pmatrix}$$

$$\begin{pmatrix} 4915 \\ 6 \end{pmatrix}$$

Vectors

How

Ordered set of numbers that represent ~~objects~~ words



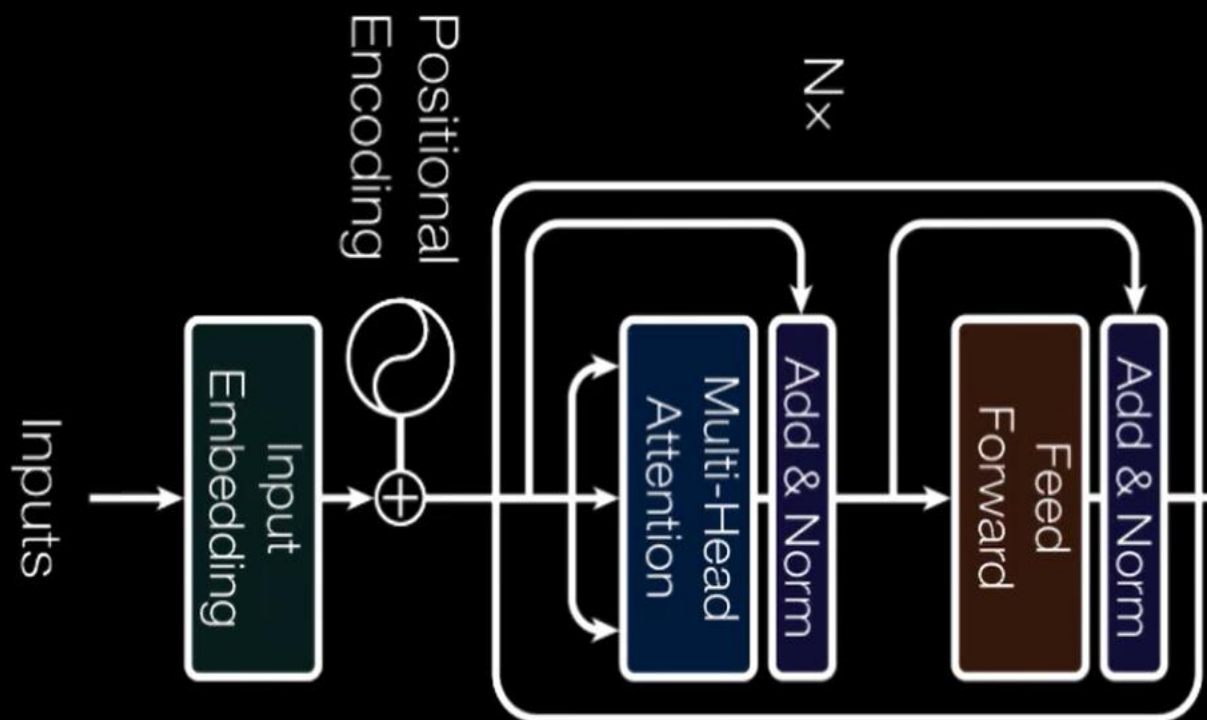
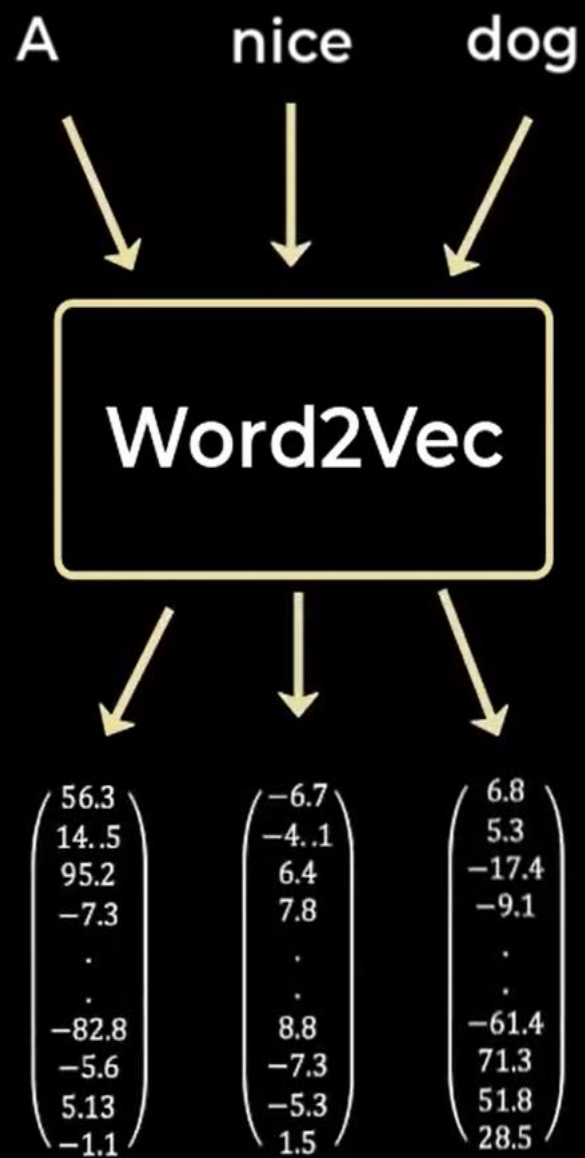
Vectors

How

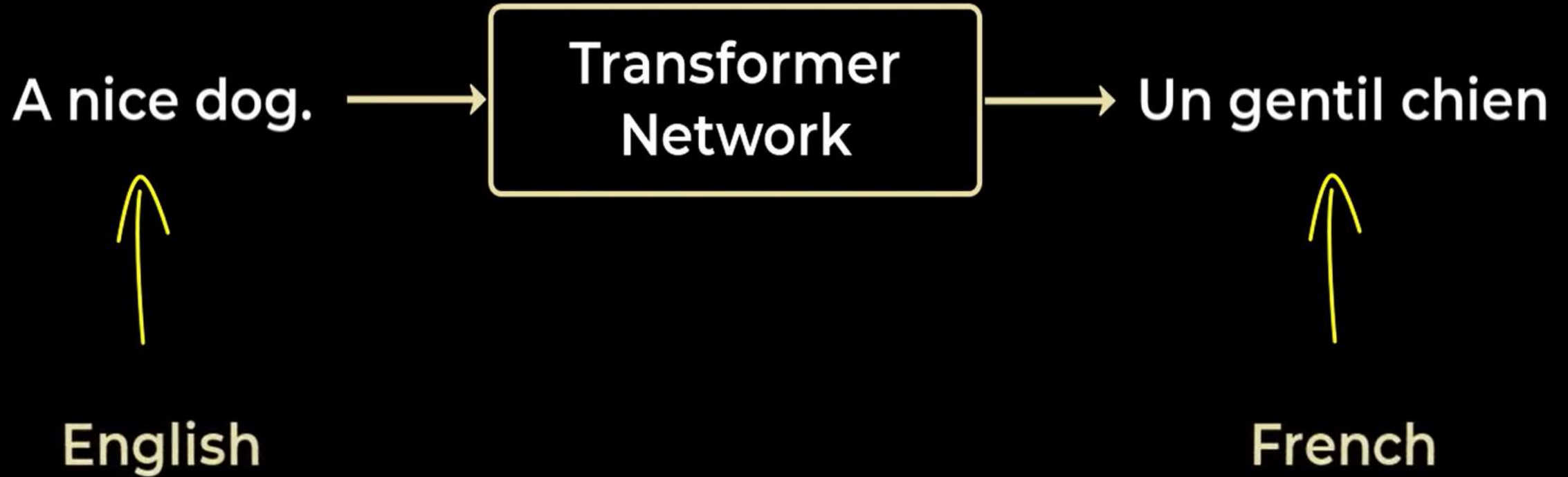
Ordered set of numbers that represent ~~objects~~ words



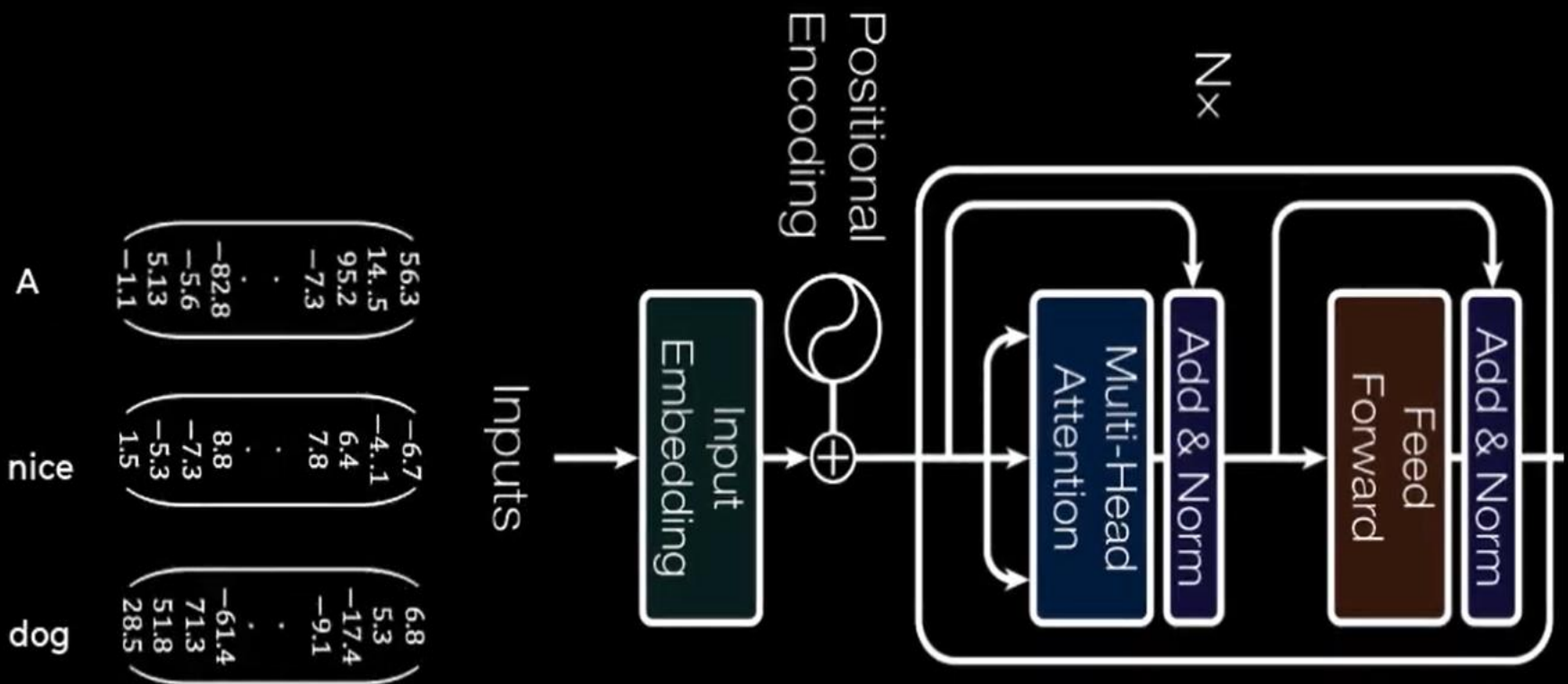
Vectors in Transformers



Transformer Networks



Vectors in Transformers



Vectors in Transformers

The **bank** of the river

Get money from the **bank**

Word2Vec

Word2Vec

$\begin{pmatrix} 31.5 \\ -2.5 \\ 9.2 \\ 7.8 \\ \cdot \\ \cdot \\ 6.8 \\ 9.4 \\ -15.3 \\ -5.1 \end{pmatrix}$

Positional Encoding

	King	on	horse
Position Embedding	0.33	0.67	1.00
Word Embedding	68	37	40
Word Embedding into Network	68.33	37.67	41
	Horse	on	King
Position Embedding	0.33	0.67	1.00
Word Embedding	40	37	68
Word Embedding into Network	40.33	37.67	69

Dictionary

King	68
Queen	70
Horse	40
Dog	30
On	37
A	17

$$\frac{\text{position of word in sentence}}{\text{sentence Length}}$$

Positional Encoding

	Horse	on	King	$\frac{\text{position of word in sentence}}{\text{sentence Length}}$
Position Embedding	0.33	0.67	1.00	
Word Embedding	40	37	68	
Word Embedding into Network	40.33	37.67	69	Meaning Retained
	Horse	on	King	$100 \times \frac{\text{position of word in sentence}}{\text{sentence Length}}$
Position Embedding	33	67	100	
Word Embedding	40	37	68	
Word Embedding into Network	73	104	168	Meaning Lost

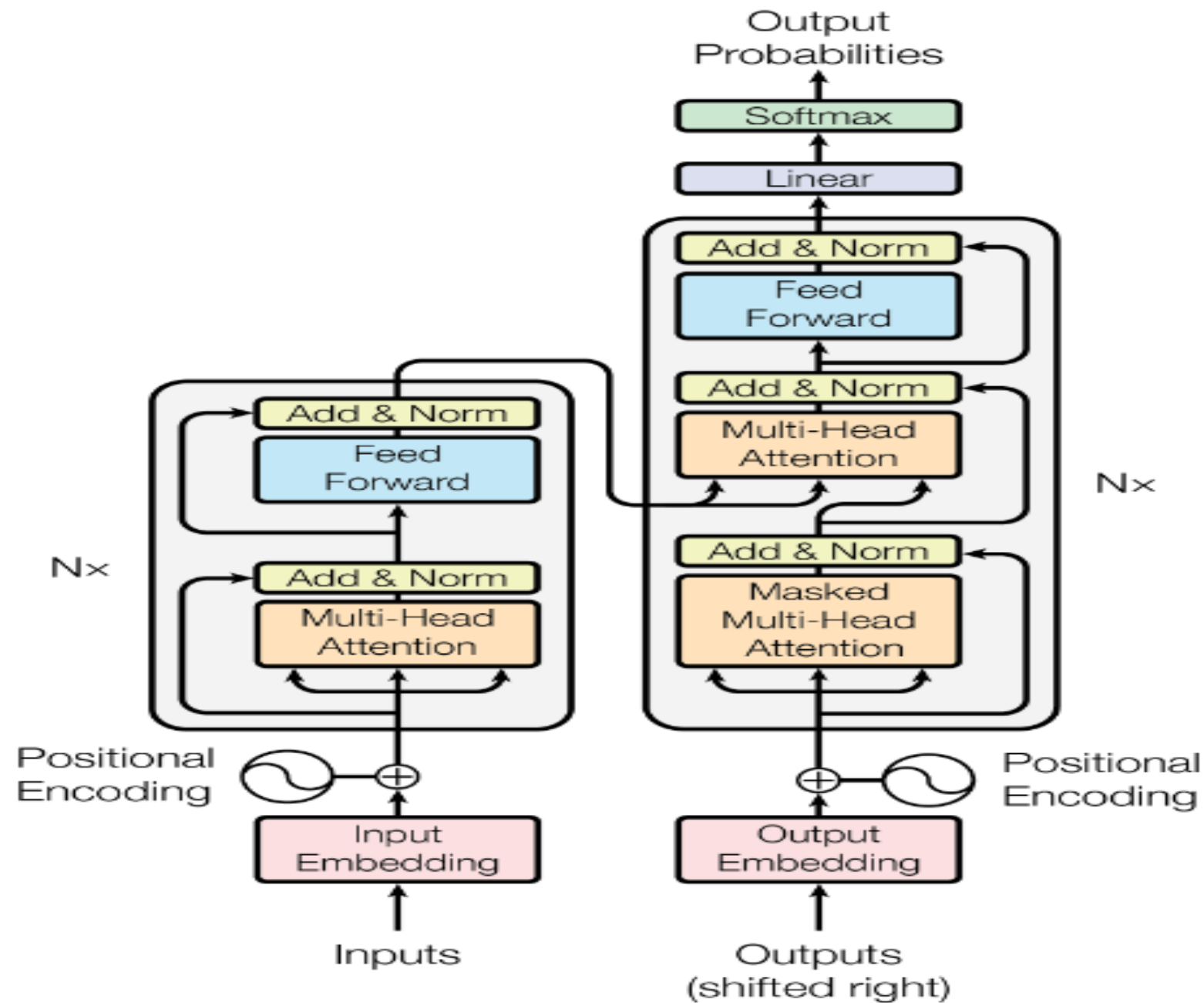


Figure 1: The Transformer - model architecture.

Vectors

What

Ordered set of numbers that represent objects


$$\begin{pmatrix} 2566 \\ 3 \end{pmatrix}$$

Square
footage of
house

Number of
Bedrooms

Vectors

Why



$$\begin{pmatrix} 2566 \\ 3 \end{pmatrix}$$



Vectors

Why



Vectors

Why

$$\begin{pmatrix} 4310 \\ 5 \end{pmatrix}$$



$$\begin{pmatrix} 2566 \\ 3 \end{pmatrix}$$

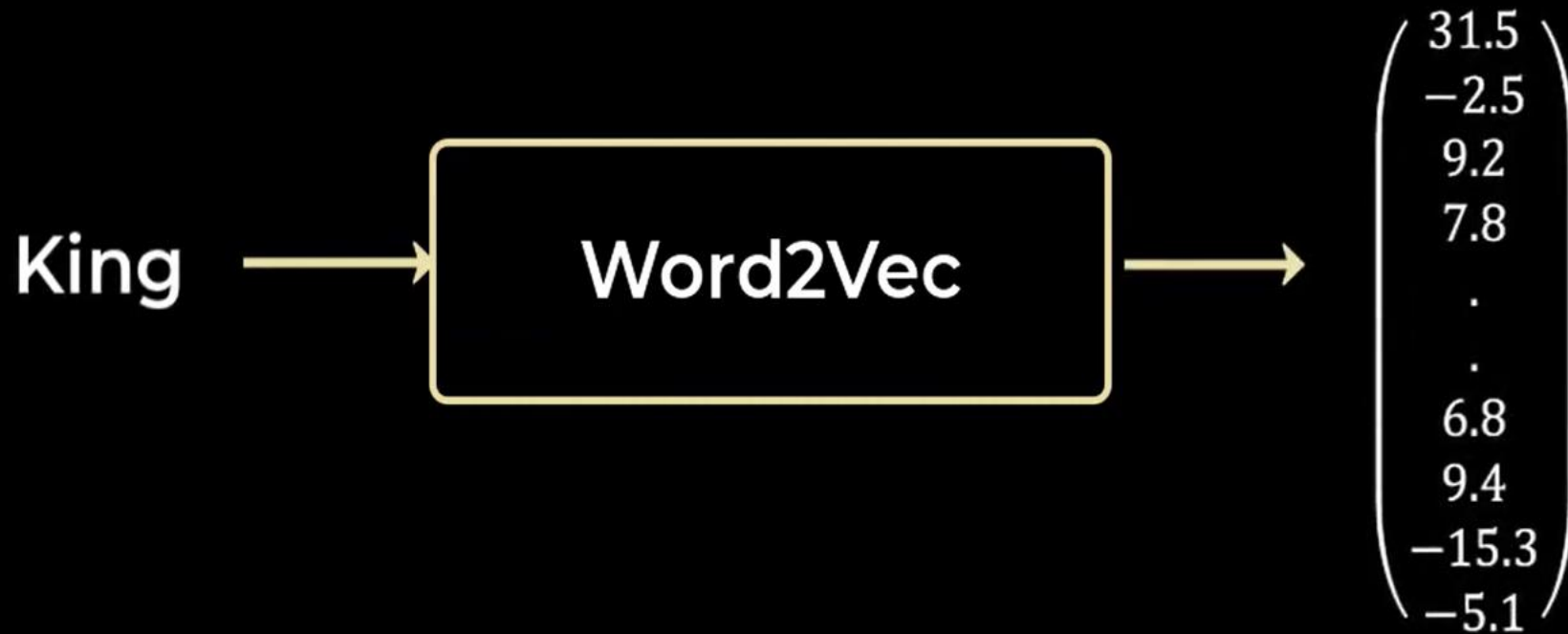
$$\begin{pmatrix} 4300 \\ 5 \end{pmatrix}$$

$$\begin{pmatrix} 4915 \\ 6 \end{pmatrix}$$

Vectors

How

Ordered set of numbers that represent ~~objects~~ words



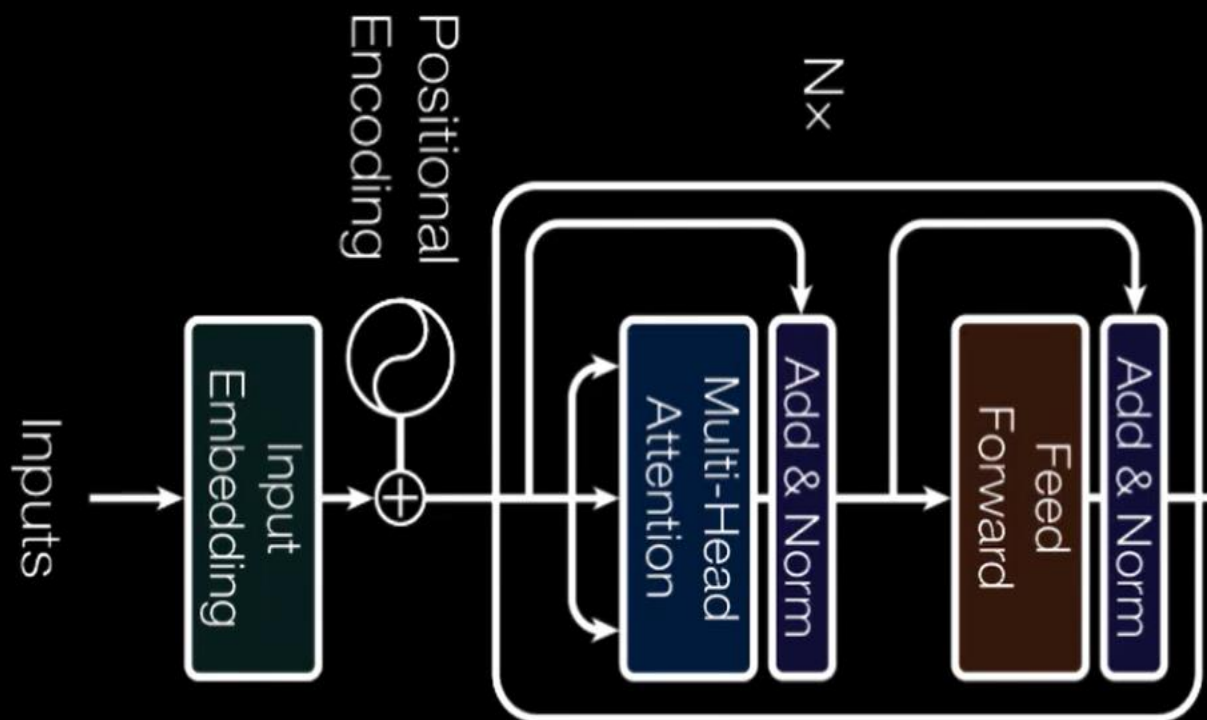
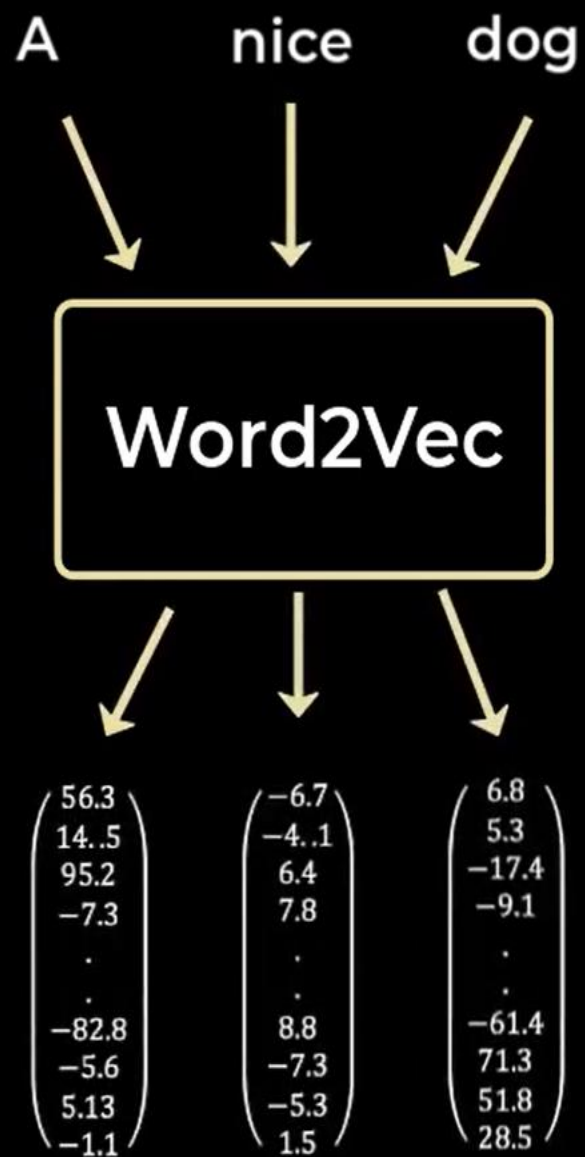
Vectors

How

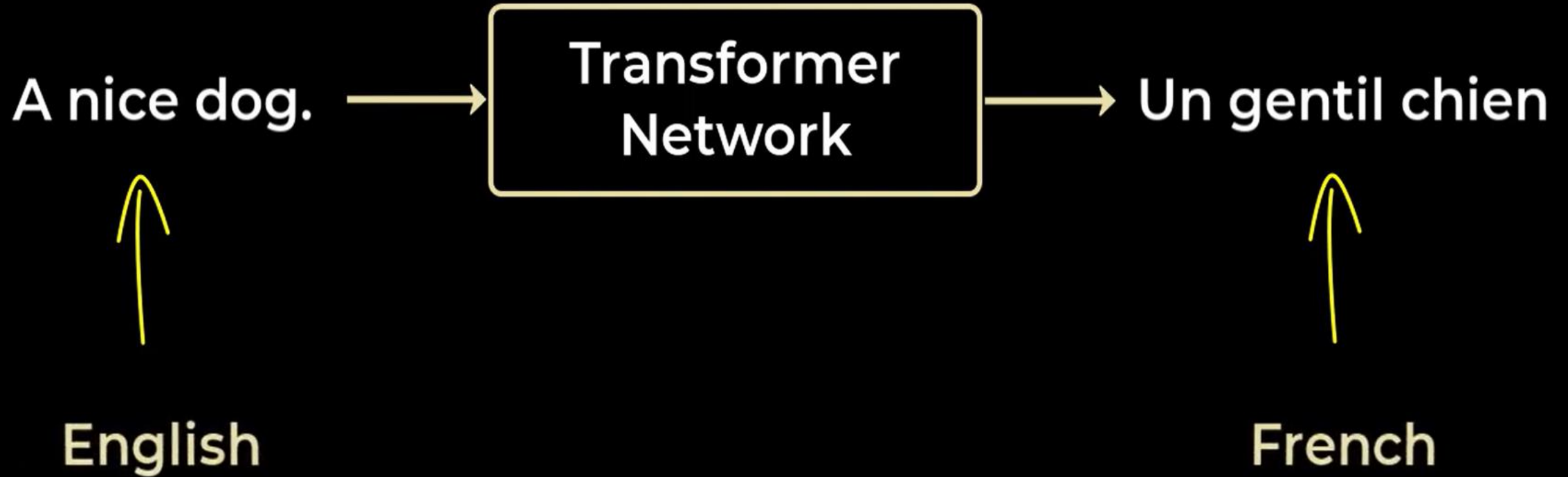
Ordered set of numbers that represent ~~objects~~ words



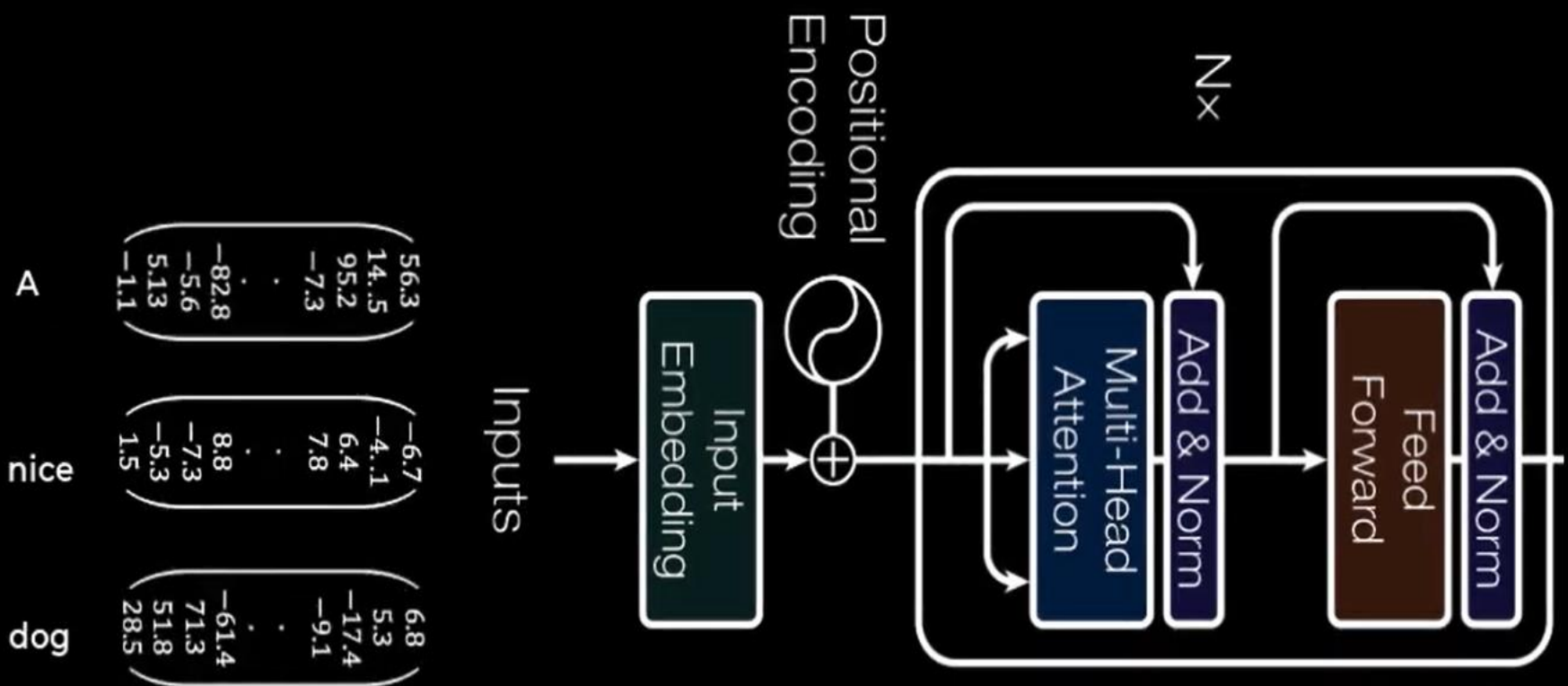
Vectors in Transformers



Transformer Networks



Vectors in Transformers



Vectors in Transformers

The **bank** of the river

Get money from the **bank**

Word2Vec

Word2Vec

$\begin{pmatrix} 31.5 \\ -2.5 \\ 9.2 \\ 7.8 \\ \cdot \\ \cdot \\ 6.8 \\ 9.4 \\ -15.3 \\ -5.1 \end{pmatrix}$

Positional Encoding

	King	on	horse
Position Embedding	0.33	0.67	1.00
Word Embedding	68	37	40
Word Embedding into Network	68.33	37.67	41
	Horse	on	King
Position Embedding	0.33	0.67	1.00
Word Embedding	40	37	68
Word Embedding into Network	40.33	37.67	69

Dictionary

King	68
Queen	70
Horse	40
Dog	30
On	37
A	17

$$\frac{\text{position of word in sentence}}{\text{sentence Length}}$$

Positional Encoding

	Horse	on	King	$\frac{\text{position of word in sentence}}{\text{sentence Length}}$
Position Embedding	0.33	0.67	1.00	
Word Embedding	40	37	68	
Word Embedding into Network	40.33	37.67	69	Meaning Retained
	Horse	on	King	$100 \times \frac{\text{position of word in sentence}}{\text{sentence Length}}$
Position Embedding	33	67	100	
Word Embedding	40	37	68	
Word Embedding into Network	73	104	168	Meaning Lost

Softmax

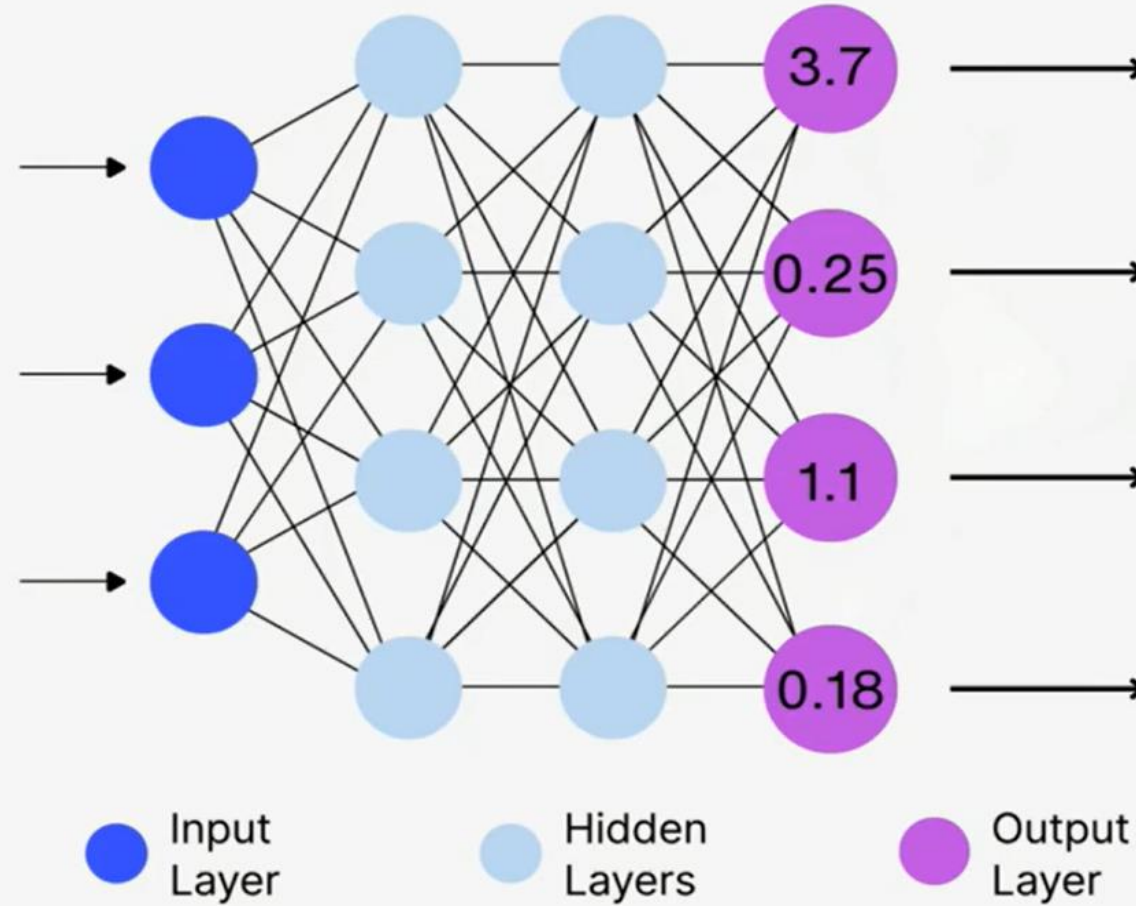
$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

where:

- z_i is the input for the current class.
- K is the total number of classes.
- e is the mathematical constant (≈ 2.71828).



Input image



$$\frac{e^{3.7}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.881$$

$$\frac{e^{0.25}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.028$$

$$\frac{e^{1.1}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.065$$

$$\frac{e^{0.18}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.026$$





Input image

$$\frac{e^{3.7}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.881$$

p(cat)=88.1%

$$\frac{e^{0.25}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.028$$

p(dog)=2.8%

$$\frac{e^{1.1}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.065$$

p(tiger)=6.5%

$$\frac{e^{0.18}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.026$$

p(rabbit)=2.6%



Softmax

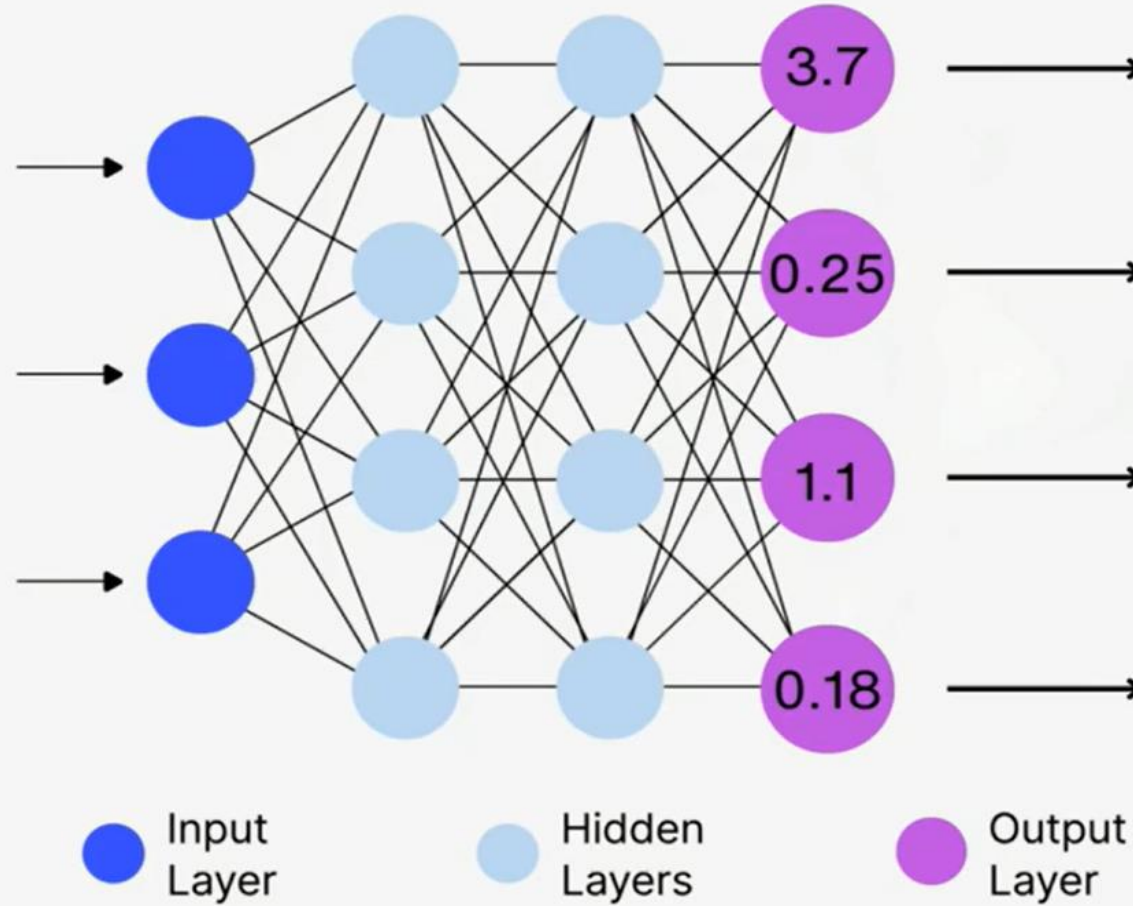
$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

where:

- z_i is the input for the current class.
- K is the total number of classes.
- e is the mathematical constant (≈ 2.71828).



Input image



$$\frac{e^{3.7}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.881$$

$$\frac{e^{0.25}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.028$$

$$\frac{e^{1.1}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.065$$

$$\frac{e^{0.18}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.026$$





Input image

$$\frac{e^{3.7}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.881$$

p(cat)=88.1%

$$\frac{e^{0.25}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.028$$

p(dog)=2.8%

$$\frac{e^{1.1}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.065$$

p(tiger)=6.5%

$$\frac{e^{0.18}}{e^{3.7} + e^{0.25} + e^{1.1} + e^{0.18}} = 0.026$$

p(rabbit)=2.6%



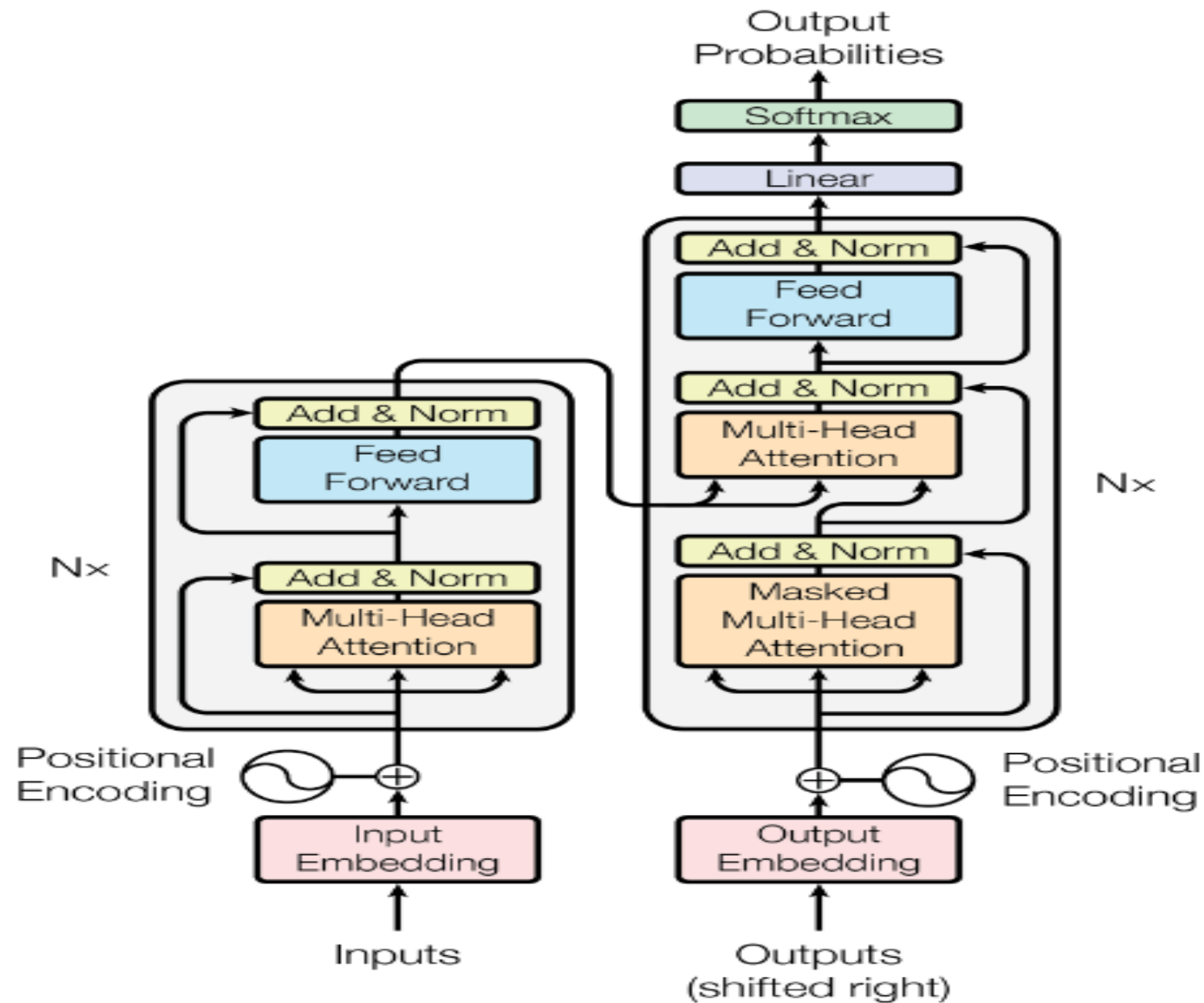


Figure 1: The Transformer - model architecture.

Self Attention

Say the following sentence is an input sentence we want to translate:

"The animal didn't cross the street because it was too tired"

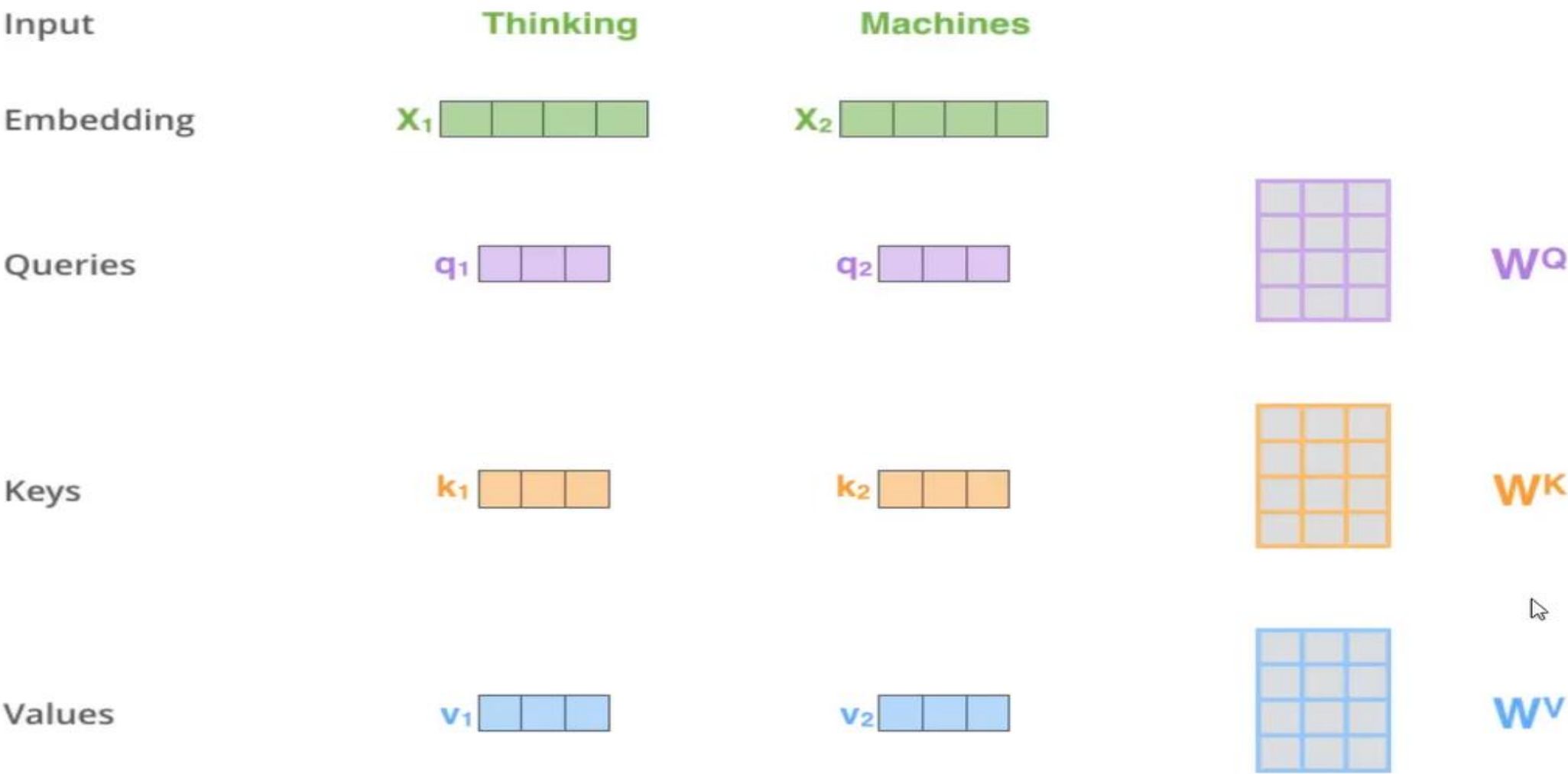
What does "it" in this sentence refer to? Is it referring to the street or to the animal? It's a simple question to a human, but not as simple to an algorithm.

When the model is processing the word "it", self-attention allows it to associate "it" with "animal".

As the model processes each word (each position in the input sequence), self attention allows it to look at other positions in the input sequence for clues that can help lead to a better encoding for this word.

Self-attention is the method the Transformer uses to bake the "understanding" of other relevant words into the one we're currently processing.

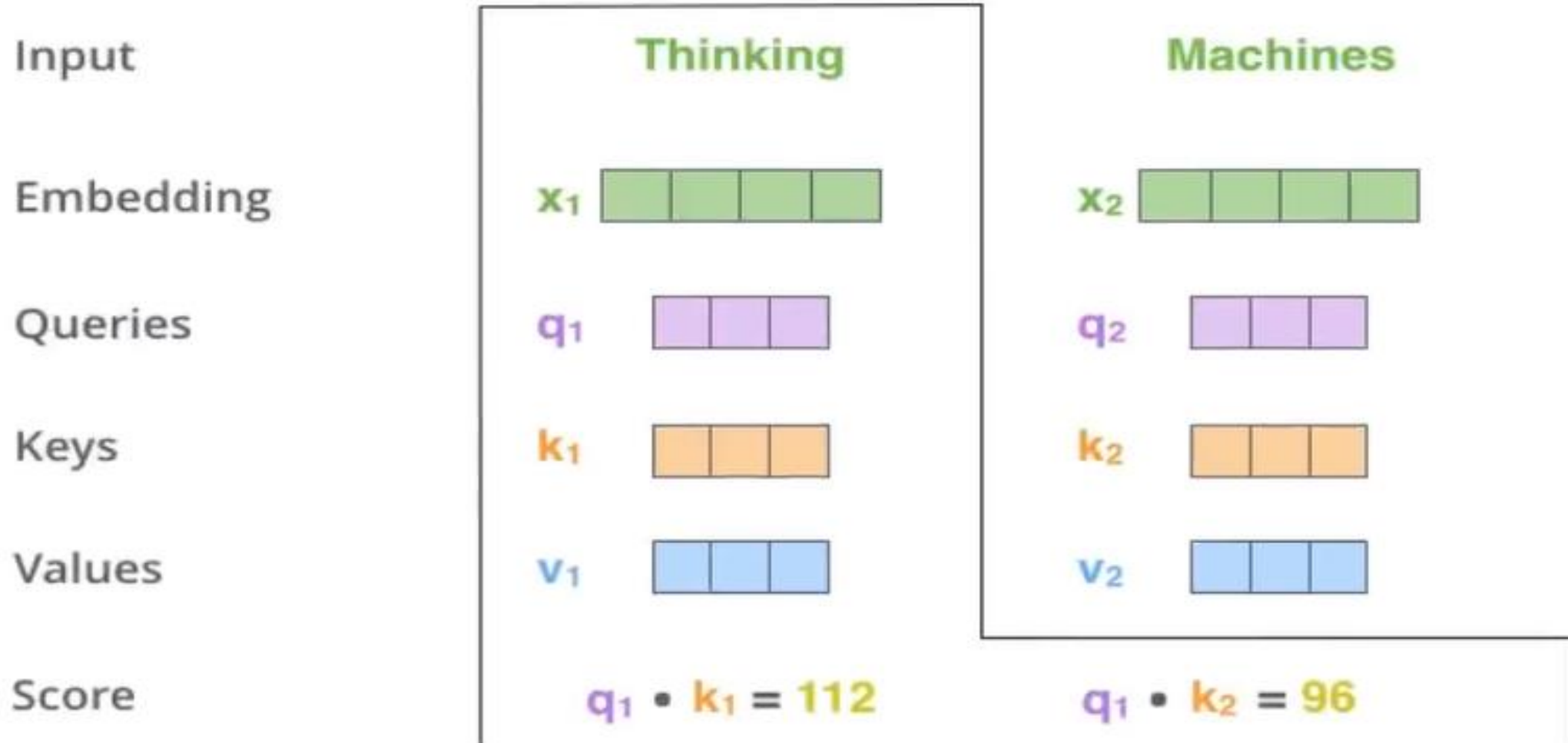
The **first step** in calculating self-attention is to create three vectors from each of the encoder's input vectors (in this case, the embedding of each word). So for each word, we create a Query vector, a Key vector, and a Value vector. These vectors are created by multiplying the embedding by three matrices that we trained during the training process.



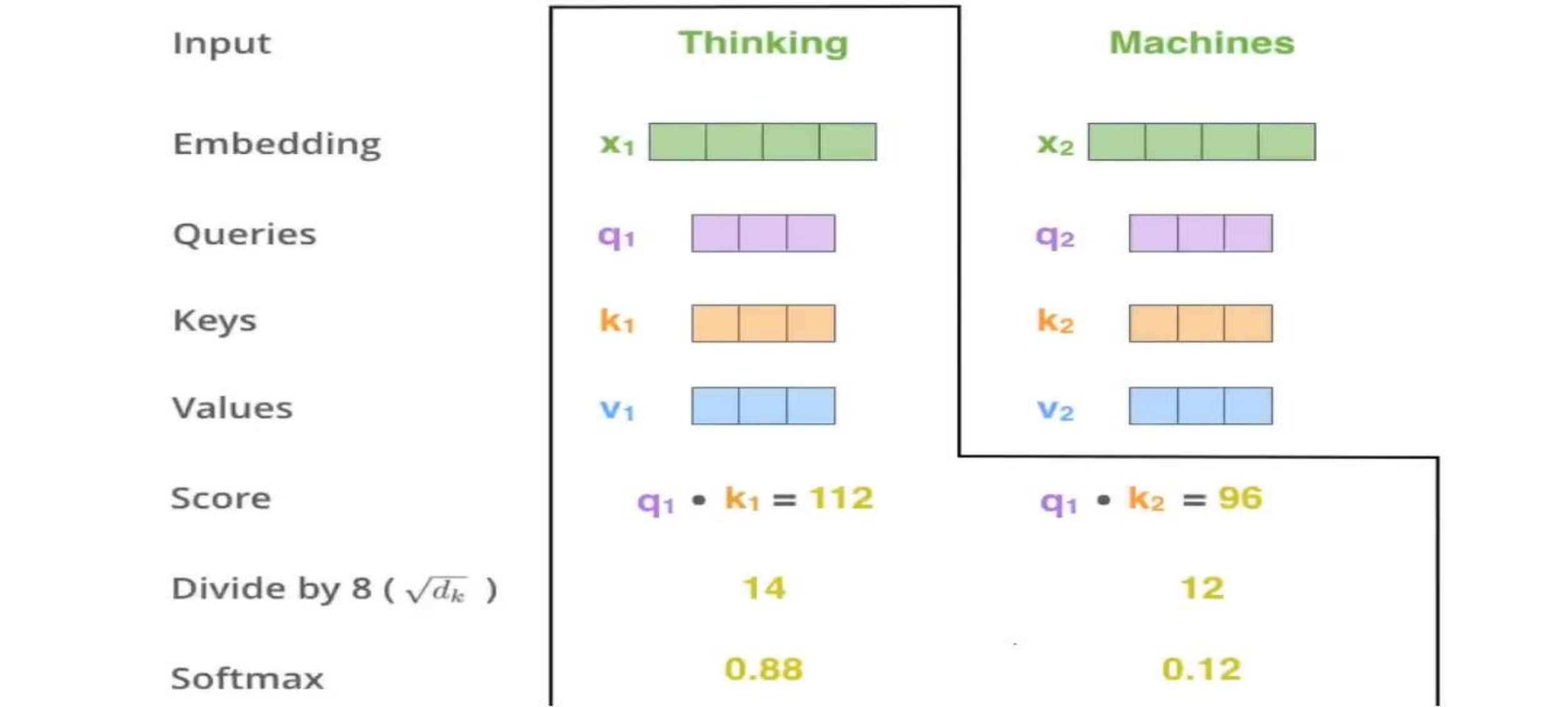
Multiplying x_1 by the W^Q weight matrix produces q_1 , the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

The **second step** in calculating self-attention is to calculate a score. Say we're calculating the self-attention for the first word in this example, "Thinking". We need to score each word of the input sentence against this word. The score determines how much focus to place on other parts of the input sentence as we encode a word at a certain position.

The score is calculated by taking the dot product of the **query vector** with the **key vector** of the respective word we're scoring. So if we're processing the self-attention for the word in position #1, the first score would be the dot product of **q1** and **k1**. The second score would be the dot product of **q1** and **k2**.

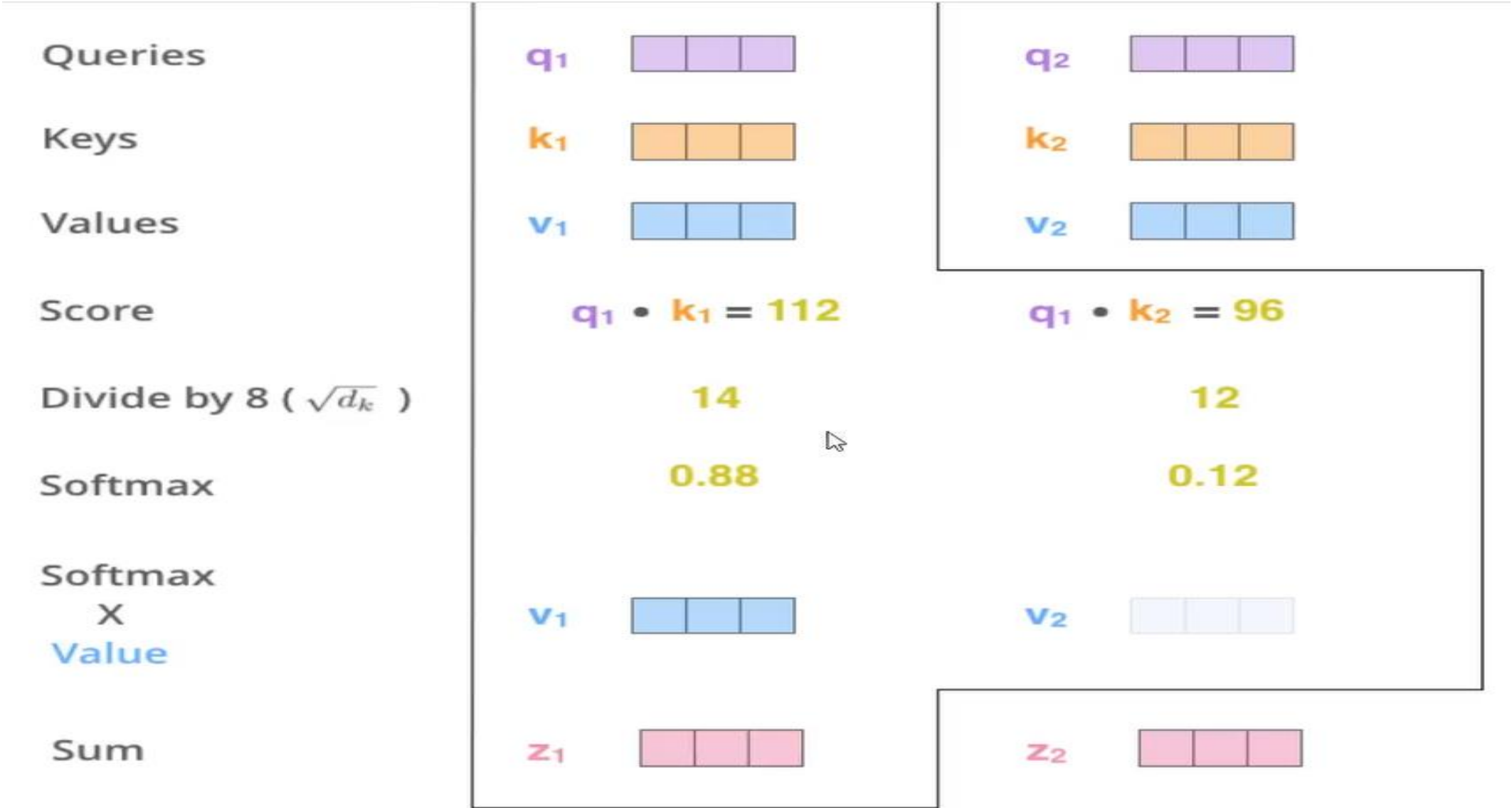


The **third and forth steps** are to divide the scores by 8 (the square root of the dimension of the key vectors used in the paper – 64. This leads to having more stable gradients. There could be other possible values here, but this is the default), then pass the result through a softmax operation. Softmax normalizes the scores so they're all positive and add up to 1.



The **fifth step** is to multiply each value vector by the softmax score (in preparation to sum them up). The intuition here is to keep intact the values of the word(s) we want to focus on, and drown-out irrelevant words (by multiplying them by tiny numbers like 0.001, for example).

The **sixth step** is to sum up the weighted value vectors. This produces the output of the self-attention layer at this position (for the first word).



Matrix Calculation of Self-Attention

The first step is to calculate the Query, Key, and Value matrices. We do that by packing our embeddings into a matrix X , and multiplying it by the weight matrices we've trained (W^Q , W^K , W^V).

$$X \times W^Q = Q$$


The diagram illustrates the calculation of the Query matrix Q . It shows a green matrix X (2 rows by 4 columns) multiplied by a purple matrix W^Q (4 rows by 4 columns) to produce a purple matrix Q (2 rows by 3 columns). The matrix X is represented by a 2x4 grid of green boxes. The matrix W^Q is represented by a 4x4 grid of purple boxes. The matrix Q is represented by a 2x3 grid of purple boxes.

$$X \times W^K = K$$


The diagram illustrates the calculation of the Key matrix K . It shows a green matrix X (2 rows by 4 columns) multiplied by an orange matrix W^K (4 rows by 4 columns) to produce an orange matrix K (2 rows by 3 columns). The matrix X is represented by a 2x4 grid of green boxes. The matrix W^K is represented by a 4x4 grid of orange boxes. The matrix K is represented by a 2x3 grid of orange boxes.

$$X \times W^V = V$$


The diagram illustrates the calculation of the Value matrix V . It shows a green matrix X (2 rows by 4 columns) multiplied by a blue matrix W^V (4 rows by 4 columns) to produce a blue matrix V (2 rows by 3 columns). The matrix X is represented by a 2x4 grid of green boxes. The matrix W^V is represented by a 4x4 grid of blue boxes. The matrix V is represented by a 2x3 grid of blue boxes.

Every row in the X matrix corresponds to a word in the input sentence. We again see the difference in size of the embedding vector (512, or 4 boxes in the figure), and the q/k/v vectors (64, or 3 boxes in the figure)

Finally, since we're dealing with matrices, we can condense steps two through six in one formula to calculate the outputs of the self-attention layer.

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$= \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

The self-attention calculation in matrix form

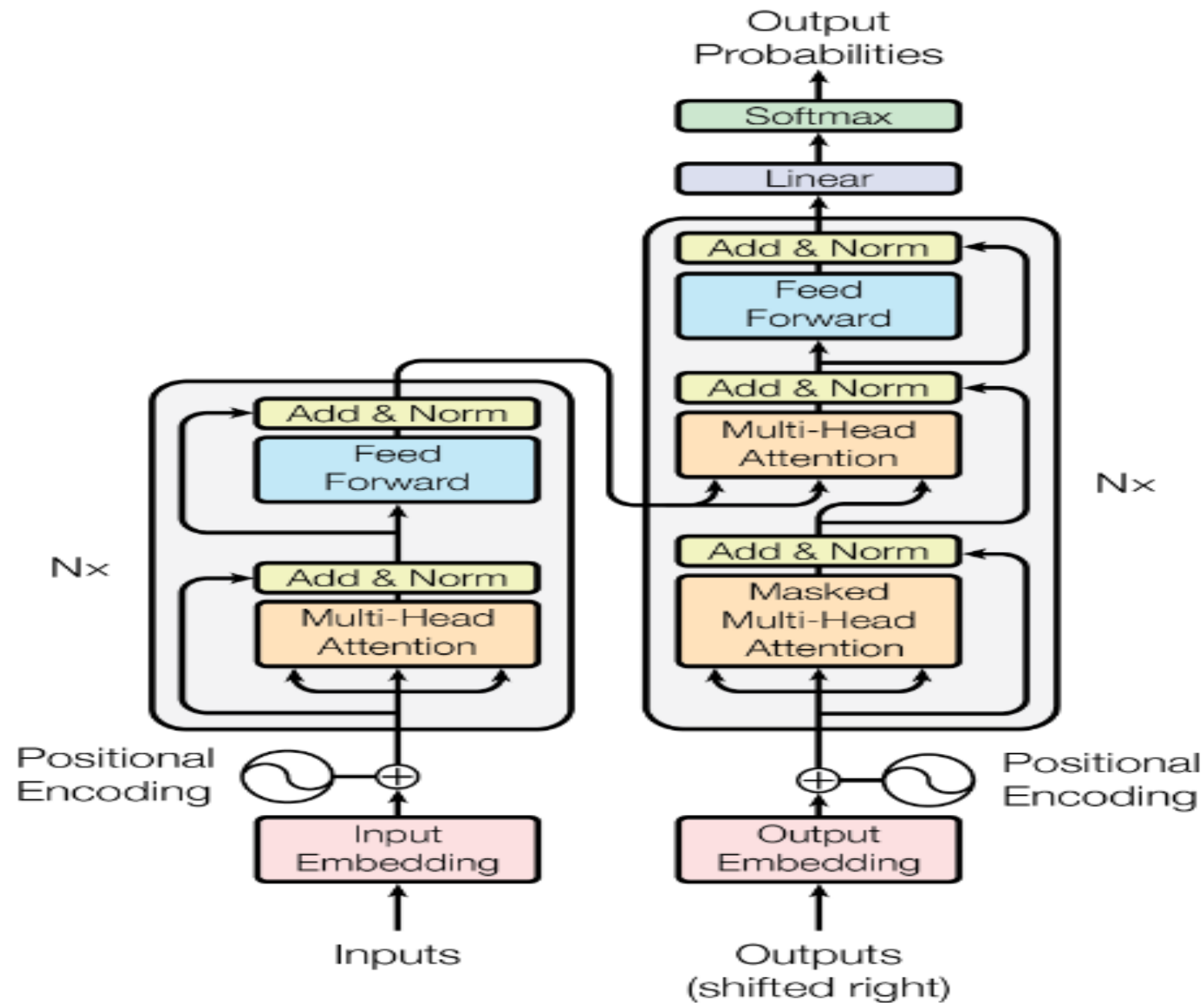


Figure 1: The Transformer - model architecture.

Matrix Calculation of Self-Attention

The **first step** is to calculate the Query, Key, and Value matrices. We do that by packing our embeddings into a matrix X , and multiplying it by the weight matrices we've trained (W^Q , W^K , W^V).

$$\begin{matrix} X \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} \times \begin{matrix} W^Q \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix} = \begin{matrix} Q \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} X \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} \times \begin{matrix} W^K \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix} = \begin{matrix} K \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} X \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} \times \begin{matrix} W^V \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix} = \begin{matrix} V \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$

Every row in the X matrix corresponds to a word in the input sentence. We again see the difference in size of the embedding vector (512, or 4 boxes in the figure), and the q/k/v vectors (64, or 3 boxes in the figure)

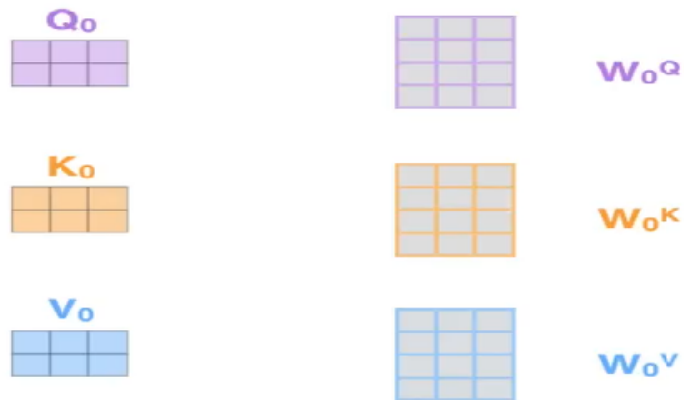
Finally, since we're dealing with matrices, we can condense steps two through six in one formula to calculate the outputs of the self-attention layer.

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$= \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

The self-attention calculation in matrix form

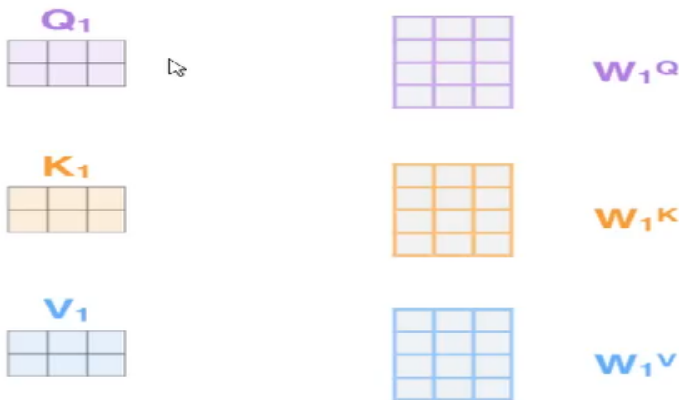
ATTENTION HEAD #0



$$\text{softmax}\left(\frac{Q_0 \times K_0^T}{\sqrt{d_k}}\right) V_0$$

$$= Z_0$$

ATTENTION HEAD #1

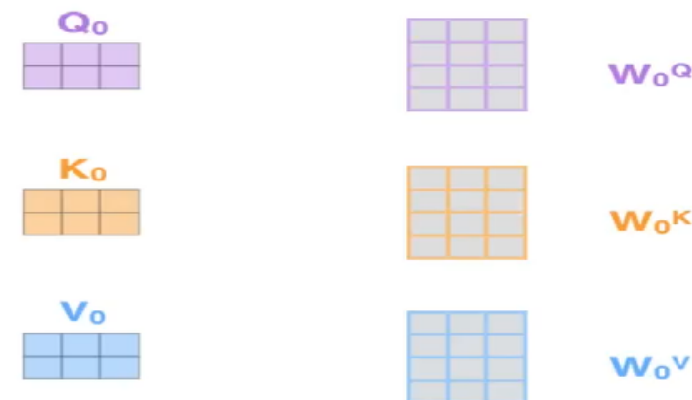


$$\text{softmax}\left(\frac{Q_1 \times K_1^T}{\sqrt{d_k}}\right) V_1$$

$$= Z_1$$

...

ATTENTION HEAD #7

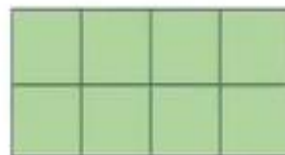


$$\text{softmax}\left(\frac{Q_7 \times K_7^T}{\sqrt{d_k}}\right) V_7$$

$$= Z_7$$

X

Thinking
Machines

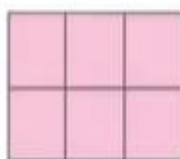


Calculating attention separately in
eight different attention heads



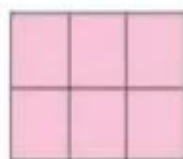
ATTENTION
HEAD #0

Z_0



ATTENTION
HEAD #1

Z_1



...

ATTENTION
HEAD #7

Z_7



1) Concatenate all the attention heads



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$

